Эволюция систем обработки больших данных

Марковский Артём Владиславович, 466636, J3310

Кейс 1. Ежедневный отчёт о продажах

Требования

Обработка является батчевой, так как полный набор данных появляется один раз в сутки, после чего его необходимо обработать за часовое окно. Потоковая обработка не требуется. Так как подготовка отчёта должна быть отделена от пользовательских запросов, нужно реализовать еще версионирование и подмену версий по готовности.

Минимальная средняя скорость чтения:

$$v_{\min} = \frac{120 * 1000}{3600} \approx 33.3\,\text{МБ/с}.$$

Но это только нижняя граница, так как будут еще накладные расходы в виде декомпрессии Parquet, фильтрацию, JOIN со справочниками, группировки, агрегации, сетевые задержки, промежуточную запись на SSD и запись результата. На деле скорость нужна больше, возможно значительно больше

Поэтому перед приобретением кластера по-хорошему измерить полное время выполнения на имеющемся сервере.

Схема архитектуры



Обоснование решения

Для обработки выбран Spark SQL в пакетном режиме. 120 ГБ данных сами по себе не доказывают необходимость распределённой системы, так как мб сервер находится близко и скорость будет высокой в общем сначала необходимо проверить фактическое время обработки.

Spark читает исходные Parquet-файлы, выполняет соединение со справочниками и формирует небольшой агрегированный результат.

Готовый результат записывается в PostgreSQL

Новая версия отчёта сначала создаётся в staging-таблице. Только после успешного завершения всего расчёта она одной транзакцией становится актуальной (можно просто давать статут типо proces, ready, failed) Исправления делаются также в целом, расчитываем заново и подменяем также одной транзакцией

Альтернативное решение

Можно юзать отдельный распределённый Spark кластер.

Это не имеет смысла так как пока что непонятно существующий сервер укладывается в часовое окно или нет. А если сделать сразу то -деньги, -время на мониторинг.

После тестов (если они будут удачные) я бы спросил у заказчика можем ли мы оставить этот временный сервер/арендовать такой же, ну а если тест неудачный, то берем кластер (подбирать мощности опять же можно исходя из времени выполнения пайплайна)

Ограничения и главный компромисс

В общем-то это решение самое простое, но может быть ничего больше и не понадобится, а если понадобится, то в целом инфра уже будет готова и нужно будет немного поменять только 1 узел. Ограничения могут возникнуть в том, что можем не успевать за час или в будущем данных будет больше.

Кейс 2. Контроль выполнения доставок

Тут уже нужна потоковая обработка, так как события приходят постоянно и панель должна обновляться постоянно

При этом обработку событий опять же нужно отделить от пользовательских запросов. Диспетчеры не должны сами ходить в сырой поток и что-то там считать, они должны читать уже готовое последнее состояние заказа.

Так как события могут приходить неравномерно и может быть такое что сразу придет куча сообщений между источниками и обработкой нужен какой-нибудь брокер сообщений типо Kafka.

Средний объём данных за сутки:

$$V = 10\,000 * 500 * 86\,400 \approx 432\,\text{ГБ/сутки}.$$

За 7 суток соответственно:

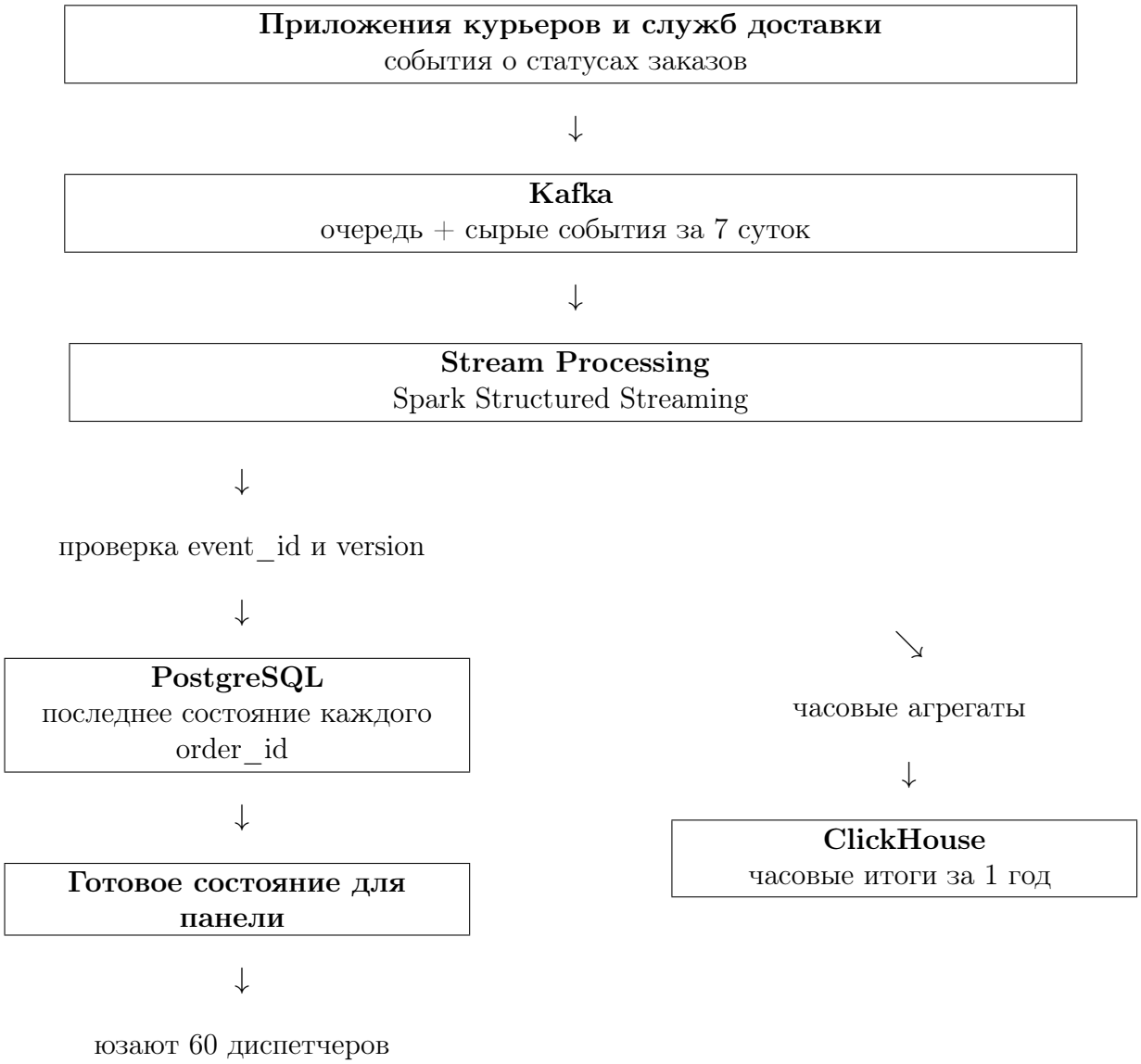
$$432 * 7 \approx 3\,\text{ТБ}.$$

Но это опять же только сами события без репликации, метаданных Kafka, индексов, логов и прочих служебных данных.

В пике входящий поток:

$$30\,000 * 500 = 15\,000\,000\,\text{байт/с} \approx 15\,\text{МБ/с}.$$

Схема архитектуры



Обоснование решения

Kafka нужна для того чтобы спокойно принимать события даже если обработка временно не успевает или после восстановления связи сразу прилетела большая накопленная пачка.

Дальше потоковый обработчик для каждого order_id смотрит version.

Например сейчас у заказа:

$$version = 4.$$

Если приходит событие:

$$version = 5,$$

то состояние обновляем. Если потом из-за задержки приходит:

$$version = 3,$$

текущее состояние им уже не заменяем.

С дубликатами примерно также. У нас есть event_id, поэтому если такое событие уже обрабатывали, второй раз ничего менять не нужно.

Если после 10 минут отсутствия связи прилетает накопленная очередь, Kafka ее принимает, после чего обработчик постепенно разгребает. Тут по-хорошему нужно чтобы максимальная скорость обработки была выше обычного среднего потока, потому что иначе после большого сбоя очередь может долго не уменьшаться.

Еще важный момент в том, что статус давно не менялся и данные давно не приходили это не одно и то же.

Например заказ может реально 60 минут ехать и иметь один и тот же статус. Поэтому кроме времени изменения статуса нужно хранить еще время когда мы в последний раз вообще видели данные от источника, для этого можно отправлять переодически статус снова, обновляя время.

Можно условно хранить:

$$last_status_change$$

и

$$last_source_seen.$$

На панели тогда можно отдельно показывать сколько времени не менялся статус и насколько свежие вообще данные от источника.

Пять секунд считаются с момента когда событие уже принято центральной системой, а не с момента когда курьер поменял статус.

То есть:

$$latency = t_{panel} - t_{accepted}.$$

Потому что по условию сам источник вообще не гарантирует что доставит событие за 5 секунд.

Проверка задержки

Проверять только скорость приёма Kafka тут не имеет смысла, потому что событие может быстро попасть в Kafka и потом еще условно 30 секунд лежать в очереди.

Поэтому:

$$\text{приём события} \rightarrow \text{Kafka} \rightarrow \text{обработка} \rightarrow \text{обновление состояния} \rightarrow \text{панель}.$$

И уже по нему смотреть чтобы:

$$p95 < 5\,\text{с}.$$

Тестировать нужно обычный поток, пик 30 000 событий/с, дубли, старые версии и накопившуюся очередь после сбоя.

Ограничения и главный компромисс

Минус в том что появляется больше инфраструктуры которую нужно поддерживать: Kafka, stream processing, PostgreSQL, ClickHouse. Вместо того, чтобы прост обрабатывать исходные данные

Еще одно ограничение в том что мы не можем гарантировать свежесть данных от самого телефона курьера, потому что сеть может отсутствовать 10 минут. Поэтому наши 5 секунд начинаются только с момента когда событие уже дошло до центральной системы

Кейс 3. Система бронирования билетов

Основные условия

Требования

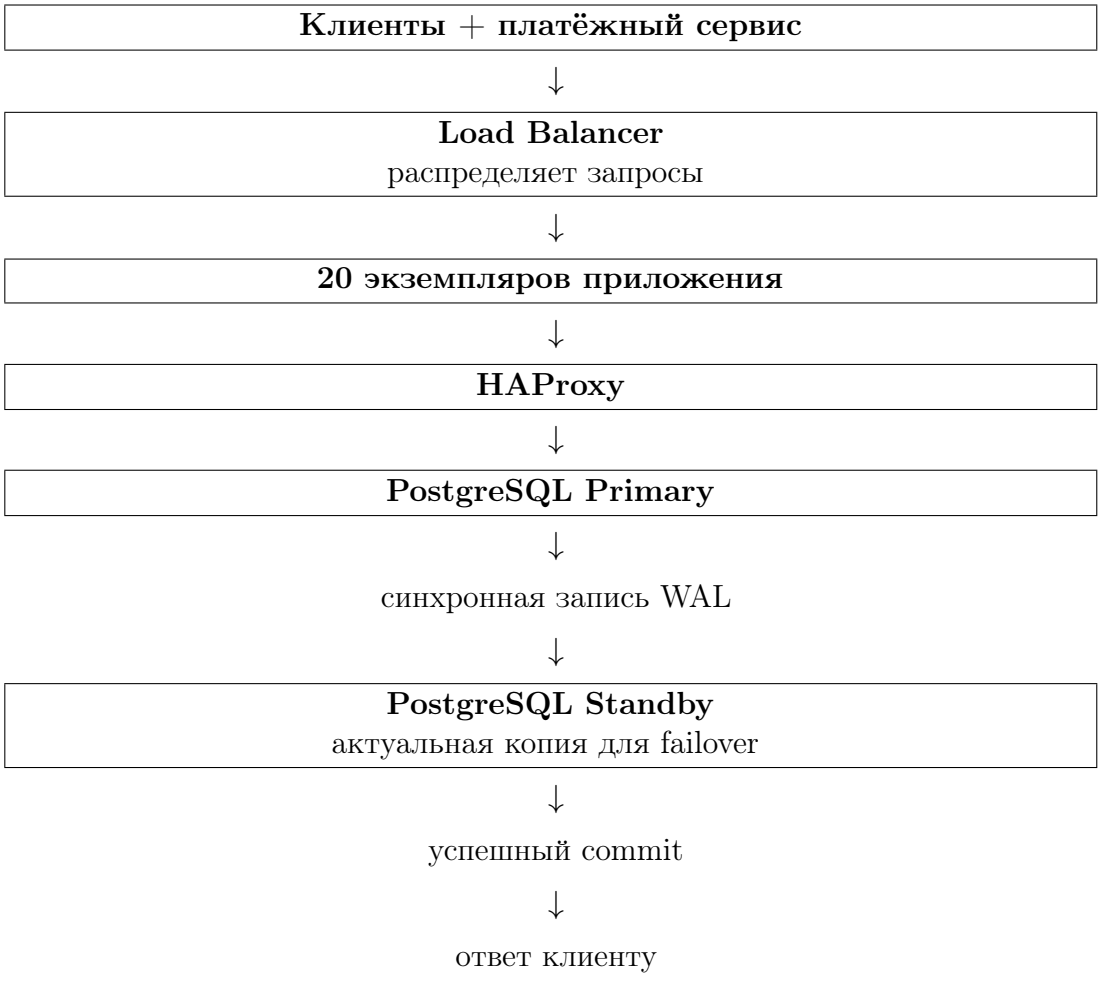
Здесь основная нагрузка операционная. Нужно сделать так, чтобы одно место ни при каких обстоятельствах не было одновременно продано двум людям.

Поэтому 20 экземпляров приложения не должны сами независимо решать свободно место или нет. Окончательное решение должно приниматься в одном согласованном хранилище.

Для этого я бы использовал PostgreSQL с транзакциями и блокировками строк.

Еще учитыват что клиент может не получить ответ и отправить ту же операцию повторно, хотя первый запрос на самом деле уже успешно выполнился.

Схема архитектуры



Обоснование решения

Если два пользователя одновременно пытаются забронировать последнее место, оба запроса могут попасть вообще на разные экземпляры приложения.

В PostgreSQL операция выполняется в транзакции.

Например первая транзакция блокирует строку нужного места:

```
SELECT ... FOR UPDATE
```

проверяет что место свободно и создаёт резерв.

Пока транзакция не закончилась второй запрос это же место изменить не может. После завершения первой транзакции второй запрос увидит уже существующий резерв и получит отказ.

Таким образом окончательное решение принимает PostgreSQL

Для резерва сохраняем время окончания:

$$expires_at = created_at + 10 \text{ минут}.$$

После этого просроченный резерв можно освобождать фоновым процессом.

Повторные запросы

Каждая операция клиента должна иметь operation_id.

При первом запросе вместе с результатом операции сохраняем этот идентификатор.

Если клиент получил тайм-аут и отправил точно такой же запрос ещё раз, система сначала проверяет operation_id.

Если операция уже была выполнена, новый резерв не создаём, а возвращаем клиенту сохранённый результат первого запроса.

То же самое нужно сделать с уведомлениями платёжной системы. Повторное уведомление об одной и той же оплате не должно второй раз менять состояние заказа.

Что означает подтверждение клиенту

Успех клиенту лучше возвращать только после того как транзакция зафиксирована в PostgreSQL и необходимые данные синхронно записаны на standby.

То есть примерно:

$$\text{запись primary} \rightarrow \text{репликация} \rightarrow \text{commit} \rightarrow \text{ответ клиенту}.$$

Если приложение умерло уже после commit, но до отправки ответа, клиент получит тайм-аут.

После этого он повторит запрос с тем же operation_id и получит уже существующий результат.

Отказ базы

Для PostgreSQL можно использовать синхронную streaming replication, а переключение primary делать например через Patroni.

При отказе текущего primary standby становится новым primary, после чего HAProxy начинает направлять запросы туда.

На время переключения новые операции могут временно не выполняться. По условию остановка до 60 секунд допустима.

При сетевом разделении лучше вообще временно отказать части операций, если нельзя гарантировать согласованную запись.

Проверка отказа

Нужно отдельно нагрузочно проверить примерно 3000 операций/с, потому что наличие 20 серверов приложения само по себе не означает что PostgreSQL выдержит такую нагрузку.

Кроме обычной нагрузки я бы проверил: два одновременных запроса на одно место, повтор одного operation_id, повтор уведомления об оплате, истечение резерва и отказ primary во время нагрузки.

После принудительного выключения primary нужно проверить что система восстановилась не больше чем примерно за 60 секунд, подтверждённые бронирования не пропали, одно место нигде не оказалось подтверждено двум пользователям и повторные запросы после восстановления возвращают прежний результат

Альтернативное решение

Можно было бы хранить блокировки мест в Redis и сначала захватывать distributed lock, а уже потом писать результат в основную базу.

Но тогда появляется еще одна распределённая система и нужно отдельно разбираться что происходит с lock при сбое Redis, сетевом разделении или переключении узлов.

При этом PostgreSQL всё равно нужен для сохранения самого бронирования.

Поэтому здесь проще сделать PostgreSQL

Ограничения и главный компромисс

Главный компромисс здесь в том что мы сознательно выбираем корректность вместо максимальной доступности.

Если PostgreSQL временно не может гарантировать безопасную запись, то часть новых бронирований лучше отклонить или поставить на паузу.

Из-за этого во время failover пользователь может несколько десятков секунд не суметь купить билет.

Зато уже подтверждённый билет не должен исчезнуть и одно место не должно быть продано дважды.

Также отдельным ограничением остаётся производительность одной согласованной PostgreSQL-системы. 80 ГБ сами по себе тут проблемой не являются, но способность выдерживать 3000 коротких конкурентных операций/с нужно будет проверять

Кейс 4. Обновление электронных ценников

Требования

В текущих условиях требование чтобы абсолютно все ценники и кассы одновременно поменяли цену ровно в 09:00 выполнить невозможно.

Только потенциально недоступных ценников может быть:

$$48\,000 \cdot 0.03 = 1\,440.$$

Некоторые из них могут оставаться недоступными еще 20 минут после начала акции.

Кроме того даже доступный ценник сначала должен получить радиокоманду, применить её и только потом отправить подтверждение.

Я бы предложил считать 09:00 временем начала применения акции, а не гарантией атомарного переключения всех 48 000 цен.

Для каждой пары магазин + товар цена считается полностью опубликованной только когда одна и та же версия подтверждена и кассовой системой, и самим электронным ценником.

При этом для периода частичного обновления нужно выбрать безопасный порядок изменения цен.

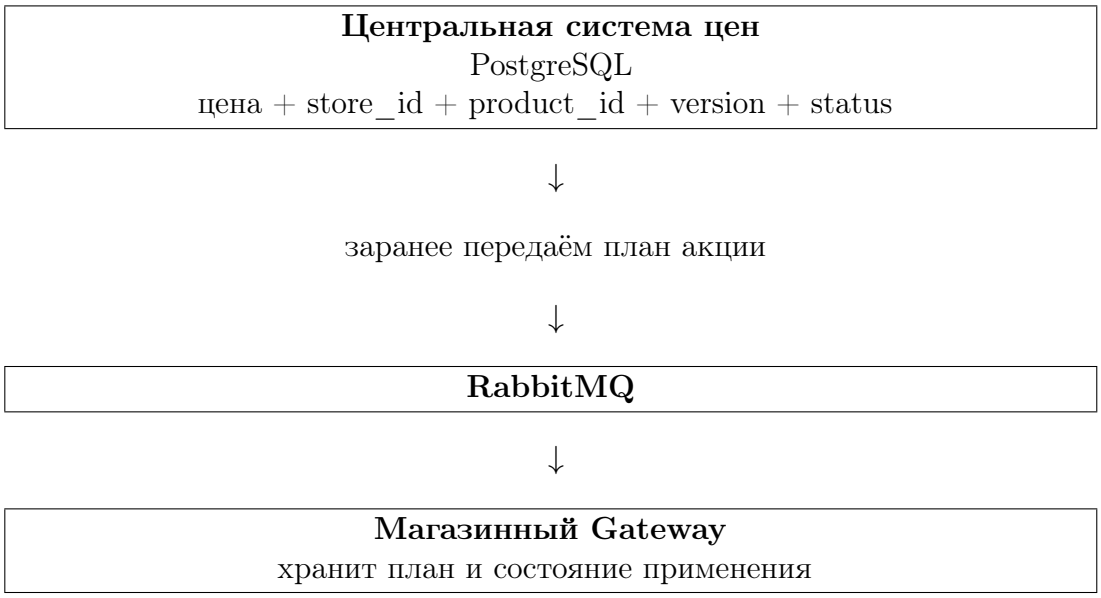
Порядок изменения цены

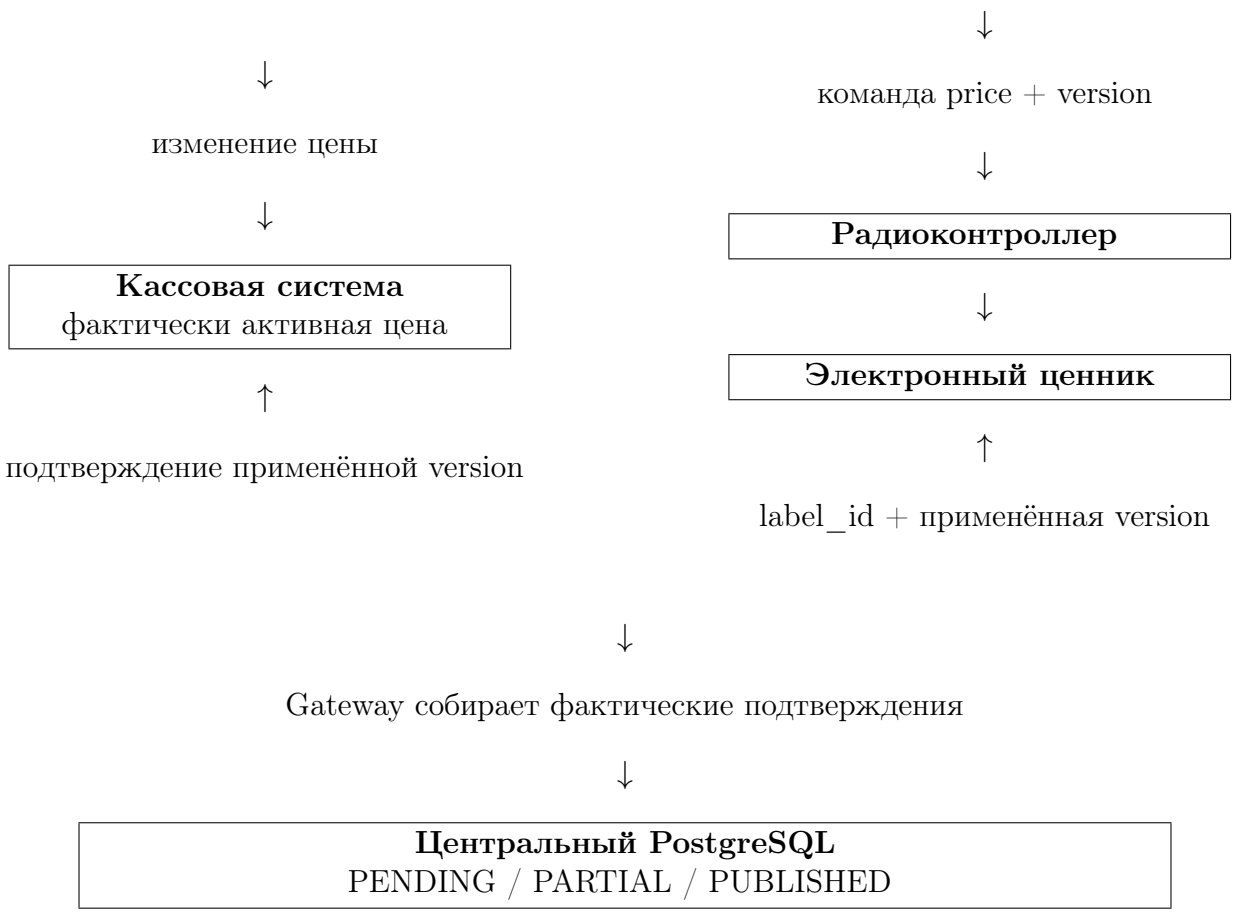
Если цена снижается, сначала безопаснее поменять кассу, а потом ценник, если цена повышается, то наоборот.

Если ценник недоступен, повышение цены на кассе для этого товара пока не активируем.

Таким образом мы жертвуем точным временем запуска отдельных цен, зато не получаем ситуацию когда покупателю на кассе внезапно выставляется цена выше той, которую он видит на ценнике и не портим лояльность покупателя (можно как аргумент использовать для заказчика)

Схема архитектуры





Обоснование решения

Центральная система хранит желаемую цену и её version для каждого магазина и товара.

Сам план акции лучше заранее передать в магазинный Gateway.

Это важно потому что связь магазина с центром может пропасть

Если план уже лежит локально, Gateway всё равно может начать его выполнять в 09:00 без связи с центральной системой.

При этом заранее отправить цену непосредственно на сам электронный ценник и попросить его переключиться потом нельзя, потому что по условию устройства так не умеют.

То есть заранее готовим только магазинный шлюз.

Еще нужна локальная SQLitечтобы после перезапуска Gateway не забывал какие команды уже были выполнены, какие еще ожидаются и какие подтверждения нужно отправить в центр после восстановления связи.

Когда цена считается опубликованной

Ответ Gateway недостаточен.

Для конкретного товара в конкретном магазине я бы хранил например:

$$target_version,$$

$$pos_version,$$

$$label_version.$$

Цена считается опубликованной только если:

$$pos_version = label_version = target_version.$$

До этого состояние остаётся pending или partil.

Повторы и старые команды

Все команды должны содержать version.

Если Gateway получил повтор уже выполненной команды с той же version, второй раз ничего логически менять не нужно.

Если приходит старая команда:

$$version = 5,$$

когда актуальная версия уже:

$$version = 6,$$

то Gateway её не должен считать актуальной и не должен откатывать состояние назад.

Проблема может быть еще в том что старая радиокоманда уже была отправлена и пришла до ценника с большой задержкой.

Если после этого ценник подтверждает старую version, Gateway видит что:

$$label_version < target_version$$

и снова отправляет последнюю актуальную цену.

Центральная система при этом также не откатывает свой статус по запоздавшему подтверждению.

Работа без связи с центром

Если план акции был заранее получен, магазин продолжает работать полностью локально.

Gateway в 09:00 начинает выполнять команды, получает подтверждения от кассы и ценников и сохраняет результаты в SQLite.

Когда связь появляется, накопленные фактические статусы отправляются в центр.

Если же магазин вообще не успел получить план акции до потери связи, применить неизвестные ему цены он естественно не сможет.

Поэтому перед 09:00 в центральной системе должен быть отдельный статус готовности магазина, например prepared или что-то подлобное, означающий что Gateway уже получил весь план акции.

Пример частичного обновления

Допустим в магазине цена товара должна снизиться:

$$100 \to 80.$$

В 09:00 касса успешно применяет:

$$version = 12,$$

но сам ценник оказывается недоступен и продолжает показывать:

$$version = 11.$$

Тогда система видит:

$$pos_version = 12,$$

$$label_version = 11.$$

И состояние товара становится PARTIAL.

На панели центральной системы сразу видно какой именно магазин, товар и ценник не совпадают.

Gateway продолжает повторять доставку новой версии.

Когда ценник наконец подтверждает:

$$label_version = 12,$$

получаем:

$$pos_version = label_version = target_version = 12,$$

и состояние можно перевести в publish.

Для повышения цены ситуация будет немного другой, пока ценник не подтвердил новую версию, касса остаётся на старой цене.

Альтернативное решение

Можно было бы в 09:00 просто разослать всем магазинам новые цены и считать изменение успешным сразу после ответа Gateway. Так было бы сильно проще. Но этот ответ говорит только о том что команда дошла до Gateway или радиоконтроллера, он вообще не гарантирует что нужная цена реально появилась на экране. При 3% недоступных устройств центральная система тогда показывала бы что акция полностью запущена, хотя потенциально больше тысячи ценников всё еще показывают старые цены. Поэтому нужен именно обратный путь фактических подтверждений.

Ограничения и главный компромисс

Главное ограничение здесь физическое: существующее оборудование не умеет атомарно переключить кассу и ценник в одну и ту же миллисекунду и не гарантирует доступность всех устройств к 09:00.

Поэтому одновременно выполнить требования все требования невозможно

Мы жертвуем точностью общего момента переключения и допускаем состояние partial для этого.

Взамен система точно знает фактическое состояние каждого изменения и выбирает порядок обновления так чтобы во время перехода покупателю не выставлялась цена выше показанной на ценнике.

Если заказчик всё-таки требует абсолютно одновременного переключения всех цен в 09:00 без переходного состояния, тогда нужно уже менять требования к оборудованию